Count Words With a Prefix

Given a Trie that stores lowercase English words, write a function:

```
int countWordsWithPrefix(TrieNode* root, string prefix);
```

This function should return the number of words in the trie that start with the given prefix. You must use a recursive helper function.

Helper function:

```
int countWordsRec(TrieNode* node);
```

Code

```
#include <string>

using namespace std;

class node {
public:
    node* children[256] {};
    bool marked = false;
};

int countWordsRec(node* n)
{
    if (n == nullptr)
        return 0;

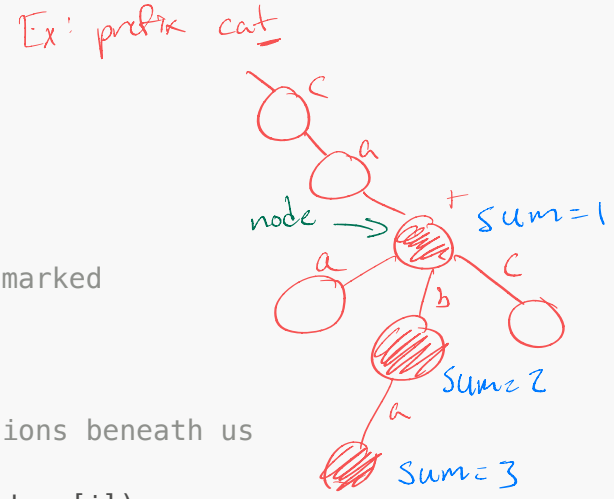
    // counter
    int sum = 0;

    // incrememnt counter if we are marked
    if (n->marked)
        sum++;

    // recursively count the completions beneath us
    for (int i = 0; i < 256; ++i)
        sum += countWordsRec(n->children[i]);

    // return the number of completions
    return sum;
}

int countWordsWithPrefix(node* root, string prefix)
{
```



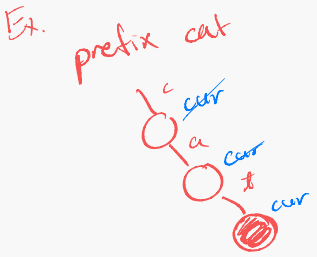
```
    if (root == nullptr)
        return 0;

    node* cur = root;

    // loop through our trie to get to a point in the trie where the prefix
is
    for (int i = 0; i < prefix.length(); ++i)
    {
        if (!cur->children[prefix[i]])
            return 0;

        cur = cur->children[prefix[i]];
    }

    // when we find the location of the prefix, call the recursive function
to count
    // the number of completions beneath our node and return that value
    return countWordsRec(cur);
}
```



Longest Word With a Prefix

Write a function:

```
string longestWordWithPrefix(TrieNode* root, string prefix);
```

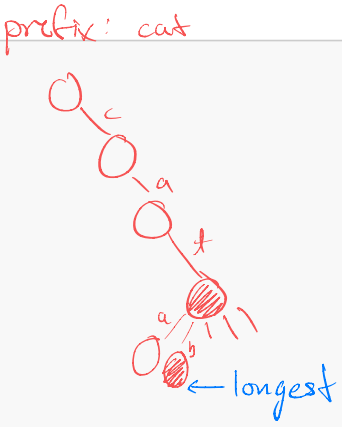
It should return the longest word in the trie that starts with `prefix`. If multiple words tie, return any one.

You must use a recursive helper.

Helper:

```
void longestWordRec(TrieNode* node, string current, string& longest);
```

Code




```
#include <string>

using namespace std;

class node {
public:
    node* children[256] {};
    bool marked = false;
};

void longestWordRec(node* n, string current, string& longest)
{
    // base case: node is nullptr
    if (n == nullptr)
        return;

    // are we at the end of a word and is it longer than the one we've
already found
    if (n->marked && current.length() > longest.length())
        longest = current;

    for (int i = 0; i < 256; i++)
    {
        if (n->children[i])
            longestWordRec(n->children[i], current + char(i), longest);
    }
}

string longestWordWithPrefix(node* root, string prefix)
{
    if (root == nullptr)
```

```
            return "";

        node* cur = root;
        string res = "";

        // loop through our trie to get to a point in the trie where the prefix
    is
        for (int i = 0; i < prefix.length(); i++)
        {
            if (!cur->children[prefix[i]])
                return "";

            cur = cur->children[prefix[i]];
        }

        // when we find the location of the prefix, call the recursive function
        // to find the longest completion of our prefix and return it
        longestWordRec(cur, prefix, res);
        return res;
    }
```

Dr. Whylie's Priority Queue

Suppose Dr. Whylie comes up to you and claims that he has invented a super-fast comparison-based priority queue. The speeds of the priority queue operations are as follows, where `n` is the number of items currently in the priority queue:

a. Insert a new item in `O(√(log n))` time. b. Extract (remove and return) the smallest item from the priority queue in `O(√(log n))` time.

Is this possible for a comparison-based priority queue? If so, provide a data structure that achieves these running times. If not, prove that it is impossible.

Explanation

No, this is not possible for a comparison-based priority queue.

Here is how we could use Dr. Whylie's priority queue to sort `n` numbers:

1. Insert all `n` numbers into the priority queue.
2. Extract the smallest item `n` times.

The extracted items come out from smallest to largest, so this would sort the numbers.

Each insert and each extract is claimed to take `O(sqrt(log n))` time. There are `n` inserts and `n` extracts, so the total sorting time would be:

```
O(n sqrt(log n))
```

But every comparison-based sorting algorithm needs at least:

```
Omega(n log n)
```

time in the worst case. Since `sqrt(log n)` grows more slowly than `log n`, `n sqrt(log n)` is asymptotically smaller than `n log n`. In other words, Dr. Whylie's queue would let us sort faster than the proven lower bound for comparison sorting.

That is a contradiction. Therefore, a comparison-based priority queue cannot guarantee both insert and extract-min in `O(sqrt(log n))` time.